



NETWORK FINGERPRINT GENERATOR & WEBSITE BEHAVIOR PROFILER

Technical Project Report

Submitted By

Muhammad Salman

Roll No: BSEF24A004

Bachelor of Software Engineering

Department of Computer Science

Session: 2024 – 2028

Submission Date: May 2026

1.Executive Summary

This report documents the complete design, development, and testing of the Network Fingerprint Generator and Website Behavior Profiler-a web-based educational tool built for networking students. The system captures live network traffic when a user visits a website,processes the raw packets through a multi-stage analytical pipeline, and produces a structured network fingerprint that summarizes how the site communicates with the internet.

The tool addresses a practical gap in networking education: students typically study protocols and traffic patterns in theory,yet rarely observe real, quantified differences between how distinct websites behave. By allowing side-by-side comparison of two websites, this project makes those differences concrete and visual.

The backend is implemented entirely in Python using Flask as the REST API server and Scapy for packet-level traffic capture. The frontend is a single-page application using vanilla JavaScript and Chart.js. All five core modules-capture, extract, fingerprint, classify, and the Flask application controller-are independently testable and loosely coupled through well-defined JSON interfaces.

2.Project Overview & Objectives

2.1 Background

Modern websites exhibit measurably different network behaviors. A video-streaming service transfers large payloads at high volume, while a static news site performs a handful of small HTTP requests and then goes idle. A social-media platform contacts dozens of unique servers and mixes protocols heavily, whereas an API-only backend sends many rapid small HTTPS packets. These differences are invisible to students reading textbook diagrams.

This project makes those differences observable by capturing real packets and encoding them into a compact,comparable fingerprint. The fingerprint is the central artifact: a structured JSON document containing protocol distributions, packet-size statistics, DNS activity, and a labeled behavioral category.

2.2 Objectives

- Capture live network traffic scoped to a specific target URL using Scapy on the active network interface.
- Extract statistically meaningful features from raw packets: sizes, protocols, inter-arrival times, unique hosts, and DNS queries.
- Assemble a structured network fingerprint JSON that summarizes all extracted features.
- Classify each fingerprint into one of four behavioral categories-Streaming,Social Media,Static Content, or API-Heavy-using a rule-based scoring engine.
- Enable side-by-side comparison of two websites with a diff report and overlay charts.

Visualize results in the browser using Chart.js: protocol pie charts,packet-size histograms,and traffic-over-time line graphs.

3.System Architecture

3.1 High-Level Architecture

The system is divided into two tiers: a Python/Flask backend responsible for all data collection and processing,and an HTML/CSS/JavaScript frontend responsible for user interaction and visualization.The two tiers communicate exclusively through REST API calls that exchange JSON.

Layer	Component	Responsibility
Frontend	index.html+style.css	Single-page UI: URL input, stepper, status, results display
Frontend	app.js	AJAX calls, DOM manipulation, Chart.js rendering
Backend	app.py (Flask)	Route handling,orchestration,DNS resolution,diff building
Backend	capture.py	Scapy packet sniffing, synthetic fallback mode
Backend	extract.py	Feature computation from raw packet list
Backend	fingerprint.py	JSON fingerprint assembly
Backend	classify.py	Rule-based behavioral classification

3.2 Project Directory Structure

The codebase is organized into a clear module hierarchy matching the README specification:

```
netfp/
├── app.py                # Flask application & API routes
├── backend/
│   ├── capture.py       # Scapy packet capture module
│   ├── extract.py       # Feature extraction module
│   ├── fingerprint.py   # Fingerprint assembly module
│   └── classify.py       # Rule-based behavior classifier
├── templates/
│   └── index.html       # Single-page UI template
├── static/
│   ├── css/style.css    # Dark / light theme stylesheet
│   └── js/app.js        # Frontend JavaScript
├── requirements.txt
└── README.md
```

4. Data-Flow and System Workflow

4.1 End-to-End Workflow

The following sequence describes the complete flow of data from user input to rendered results, matching the eight steps defined in the project specification:

1. User enters a target URL (and optionally a comparison URL) in the browser and clicks Start Analysis.
2. The frontend JavaScript validates the URL format client-side before sending a POST request to `/api/analyze` or `/api/compare`.
3. Flask receives the request, calls `resolve_host`3. Flask receives the request, calls `resolve_host()` which uses `socket.getaddrinfo()` to obtain all IP addresses for the domain.
4. `capture.py` launches a background thread that hits the target URL repeatedly with HTTP requests to generate traffic while Scapy sniffs the interface with a BPF filter scoped to the resolved IPs.
5. After the configurable capture window expires (default 10 s), the raw packet list is passed to `extract.py`.
6. `extract.py` normalizes Scapy objects into uniform dicts and computes: total packet count, protocol distribution, packet sizes, inter-arrival times, unique destination IPs, DNS query names, total bytes, and a bytes-per-second timeline.
7. `fingerprint.py` packages all feature values into a labelled JSON fingerprint and calls `classify.py` to append a `behavior_label` and confidence score.
8. Flask returns the fingerprint JSON to the frontend, which populates the summary card and renders three Chart.js charts.

4.2 Data Transformation Pipeline

Each module performs a specific, bounded transformation:

Stage	Input	Output
<code>capture.py</code>	URL, resolved IPs, duration	List of packet objects (Scapy or synthetic dicts)
<code>extract.py</code>	Packet list	Feature dictionary with all statistics
<code>fingerprint.py</code>	Feature dict + URL	Structured fingerprint JSON
<code>classify.py</code>	Fingerprint JSON	Behavior label + confidence integer
<code>app.py</code>	Two fingerprints	Diff object with ratios and comparisons

5.Module-by-Module Technical Design

5.1 capture.py-Packet Capture

This module is the entry point for data collection. It implements two operating modes:

Live Capture Mode

When running with root privileges and a live network interface, `capturetraffic()` launches a daemon thread that repeatedly calls `requests.get(url)` to generate real outbound traffic. Simultaneously, Scapy's `sniff()` function runs with a BPF filter of the form `(host X.X.X.X or host Y.Y.Y.Y) or port 53`, restricting capture to traffic relevant to the target domain plus all DNS. Once the timeout expires, `wrpcap()` persists the packets to a temporary `.pcap` file under the system's temp directory.

Synthetic Fallback Mode

In environments without root access-such as shared lab machines or CI/CD systems-the module generates a deterministic synthetic packet trace via `_synthetic_capture()`. The trace is seeded with `hash(hostname)`, ensuring reproducibility. Heuristics based on the hostname (keywords like 'netflix', 'youtube', 'bbc', 'api.') adjust the packet count, mean size, and protocol mix to produce realistic-looking data for each site category. This design choice ensures the rest of the pipeline remains fully exercisable without any system-level privileges.

5.2 extract.py-Feature Extraction

`extract_features()` accepts the heterogeneous packet list and first normalizes each item through `_packet_view()`, which handles both Scapy packet objects and synthetic dicts. The function then computes:

Protocol classification: Scapy packets are inspected for IP, TCP, UDP, DNS, and ICMP layers. TCP packets on ports 80/443 are relabelled HTTP/HTTPS respectively, giving a finer-grained protocol breakdown than a raw TCP count would provide.

- Size statistics: Mean, minimum, and maximum packet sizes in bytes, computed over all packets.

- Histogram buckets: Packets are grouped into five size ranges (0-100, 101-500, 501-1000, 1001-1500, 1500+ bytes), directly corresponding to the Chart.js bar chart.

- Bytes-per-second timeline: A list of length equal to the capture duration where index *i* holds the total bytes transferred during second *i*. This feeds the traffic timeline chart.

- Unique destination IPs: A sorted, deduplicated set of all dst addresses from IP-layer packets.

- DNS query names: All qname values decoded from DNS question records.

Inter-arrival times: Sorted timestamps differenced pair-wise, capped at 200 values to limit payload size.

`An_emptyfeatures()` helper returns a zero-filled dictionary for the edge case where no packets are captured, preventing downstream `KeyError` exceptions.

5.3 fingerprint.py-Fingerprint Assembly

buildfingerprint() is deliberately thin: it copies feature values into a flat, human-readable JSON structure with consistent field names and appends the capture timestamp in UTC. The behavior_label and confidence fields are seeded to 'Unknown' / 0 here and overwritten by classify.py immediately after the call, keeping the fingerprint module free of classification logic. This separation of concerns makes each module independently unit-testable.

5.4 classify.py-Behavioral Classifier

The classifier operates in two stages: strong-signature rules and fallback scoring.

Strong-Signature Rules

Four if-conditions implement high-confidence pattern matching that triggers before any scoring:

Label	Trigger Conditions	Confidence Formula
Streaming	mean_packet_size ≥900 AND total_bytes ≥ 1,500,000 AND top protocol is TCP/UDP/HTTP/HTTPS	$\min(95, 80 + \text{top_pct}/5)$
API-Heavy	mean_packet_size ≤350 AND top protocol is TCP/HTTP/HTTPS AND total_packets ≥120	$\min(92, 70 + (100 - \text{top_pct})/5)$
Social Media	unique_ips ≥12 AND dns_queries ≥4 AND top_protocol_pct < 88	$\min(90, 70 + \text{unique_ips}/2)$
Static Content	total_bytes < 1,200,000 AND total_packets < 700 AND dns_queries ≤ 6	$\min(88, 65 + (100 - \text{top_pct})/6)$

Fallback Scoring

If no strong rule matches, four scoring functions (_scorestreaming, scoreapi, scoresocial, _score_static) each return a 0-100 value based on weighted, clamped metrics. The label with the highest score wins. Confidence is derived from the gap between the top and second-best score to prevent spuriously high confidence when two categories are close.

5.5 app.py-Flask Application Controller

app.py exposes four routes:

Method	Endpoint	Request Body	Response
GET	/	-	Renders index.html via Jinja2
POST	/api/validate-url	{url}	{valid,host,ips}or error
POST	/api/analyze	{primaryUrl, captureDuration}	{siteA:fingerprint}
POST	/api/compare	{ primaryUrl, compareUrl, captureDuration}	{siteA,siteB,diff}
GET	/api/download?site=A B	-	fingerprint_A.json download

The LASTRESULTS dictionary acts as a minimal in-memory session store, persisting the most recent fingerprints so the download endpoint can serve them without re-running a capture. The builddiff() helper computes ratios and 'who has more' labels for five key metrics, driving the comparison table in the frontend.

6. Frontend Design and User Interface

6.1 Layout and Navigation

The user interface is organized as a single-page application with four primary sections presented vertically, guided by a four-step stepper that tracks analysis progress: Enter URLs → Capture Traffic → Analyze → Results. This progressive-disclosure pattern keeps the interface uncluttered while communicating system state clearly.

The header is sticky, remaining visible during scrolling. The dark theme uses CSS custom properties (CSS variables under `:root`) for all colors, allowing a complete light-mode swap by toggling the `body.light` class-implemented in 20 lines of JavaScript.

6.2 Form Validation

URL validation is performed on two levels. On the frontend, JavaScript checks that the value starts with `http://` or `https://` and that a valid hostname is present before allowing submission. The inline validation message appears below the input field without a page reload. On the backend, the `/api/validate-url` endpoint additionally attempts DNS resolution, catching domains that are syntactically valid but do not exist. This dual-layer approach avoids wasting a 10-second capture window on an unreachable URL.

6.3 Chart.js Visualizations

All three required chart types are rendered client-side after the fingerprint JSON arrives:

- Protocol Pie Chart: A doughnut chart showing the percentage breakdown of TCP, UDP, DNS, HTTP, HTTPS, ICMP, and OTHER packets. Labels display the protocol name and rounded percentage. The chart uses Chart.js 4.4 with the doughnut type and a custom legend positioned below.
- Packet Size Histogram: A bar chart grouping packets into five size buckets. Bar colors use a gradient from blue to cyan to give a visual sense of size scaling. The x-axis labels name the byte ranges; the y-axis shows packet count.
- Traffic Timeline: A line chart plotting bytes-per-second over the capture window. In compare mode, a second dataset is overlaid using a contrasting color (orange vs. blue) so students can directly see how the two sites differ in traffic volume over time.

6.4 Comparison Report

When two URLs are analyzed, the frontend renders a comparison table with five rows-Total Bytes, Total Packets, Unique IPs, Mean Packet Size, DNS Queries-plus behavior labels for each site. Cells for the higher value are styled in red and cells for the lower in green, providing an immediate visual indication of which site was more active in each dimension. A ratio string (e.g., '4.20x') quantifies the difference numerically.

7.Code Quality,Design Patterns & Best Practices

7.1 Separation of Concerns

Each backend module has exactly one responsibility and no circular dependencies. `capture.py` does not parse packets; `extract.py` does not assemble fingerprints; `fingerprint.py` does not classify.This design makes each module independently replaceable-for example, swapping out the rule-based classifier for a machine-learning model would require changing only `classify.py` and the call site in `app.py`.

7.2 Graceful Degradation

The system is designed to be fully runnable without root privileges or a live network interface. The `SCAPYAVAILABLE` flag and the `USE_LIVE_CAPTURE` environment variable allow controlled fallback to synthetic mode. This was a deliberate decision to ensure the educational tool can be demonstrated on any machine, including sandboxed environments.

7.3 Defensive Programming

All packet parsing is wrapped in `try/except` blocks that return safe zero/None defaults rather than propagating exceptions. The `_empty_features()` function prevents downstream failures when captures yield no packets. Flask routes return structured JSON error objects with meaningful messages rather than generic 500 responses. The download endpoint returns a 404 with a descriptive message if no fingerprint exists yet.

7.4 Consistent Data Contract

The fingerprint JSON schema acts as the contract between all modules. Field names are snake_case throughout,matching Python conventions on the backend and requiring minimal transformation on the JavaScript side. All numeric values are explicitly cast (`int()`, `round()`) before serialisation to prevent float precision issues in JSON output.

7.5 CSS Architecture

The stylesheet uses CSS custom properties exclusively for color values, enabling the dark/light theme toggle with a single class change. Layout is implemented with CSS Grid (`grid-4` for the input form, `charts-grid` for visualizations, `summary-grid` for fingerprint cards) to allow the UI to reflow gracefully on narrow screens. Media queries at 768 px adjust the header and container padding for mobile viewports.

8.Functional Requirements Traceability

The table below maps each functional requirement from the project specification to the implementing component(s):

FR	Requirement	Implementation	Status
FR-1	URL Input & Validation	index.html input + app.js validateUrl() +/api/validate-url	Implemented
FR-2	Packet Capture (Scapy)	capture.py-_live_capture()+_synthetic_capture()	Implemented
FR-3	Feature Extraction	extract.py-extract_features()	Implemented
FR-4	Fingerprint Generation	fingerprint.py-buildfingerprint()	Implemented
FR-5	Traffic Classification	classify.py-classify_behavior()	Implemented
FR-6	Side-by-Side Comparison	/api/compare+builddiff()+frontend comparison table	Implemented
FR-7	Chart.js Visualizations	app.js-renderProtocolChart(), renderHistogram(), renderTimeline()	Implemented

9.Testing Strategy and Validation

9.1 Unit Testing-Module-Level

extract.py Testing

The feature extractor was tested with controlled synthetic packet lists that specify exact protocol,size, and timestamp values, allowing expected outputs to be hand-calculated and compared against actual outputs. Edge cases tested include:

- Empty packet list→_empty_features() returns all-zero dict without raising exceptions.
- Single-packet list→mean,min,max all equal to that packet's size.
- All-DNS packet list→protocol_distribution shows 100% DNS, unique_ips contains only 8.8.8.8 addresses.

Mixed Scapy objects and dict inputs → _packet_view() normalizes both correctly.

classify.py Testing

The classifier was tested with five handcrafted fingerprint dictionaries representing archetypal network profiles: a streaming site (large packets, high volume), an API backend (small rapid packets), a social platform (many IPs, mixed protocols), a static blog (low traffic), and a borderline case to exercise the fallback scoring path. Expected labels and approximate confidence bounds were verified for each case.

fingerprint.py Testing

`build_fingerprint()` was tested by asserting that all required fields are present in the output dict and that `behaviorlabel` is initialized to 'Unknown' before `classify.py` overwrites it.

9.2 Integration Testing-API Endpoints

Flask's built-in test client was used to exercise the API without requiring a live network. The backend was configured with `USE_LIVE_CAPTURE=0` to force synthetic mode, enabling deterministic responses. Tests covered:

- `POST /api/analyze` with a valid URL → HTTP 200, `siteA` field present, fingerprint schema complete.
- `POST /api/analyze` with an invalid URL (missing `https://`) → HTTP 400, `error` field present.
- `POST /api/compare` with two valid URLs → HTTP 200, `siteA`, `siteB`, `diff` fields all present.
- `GET /api/download?site=A` before any analysis → HTTP 404 with descriptive message.
- `GET /api/download?site=A` after analysis → HTTP 200, `Content-Disposition: attachment`, valid JSON body.

9.3 Frontend Validation Testing

Client-side URL validation was tested manually in Chrome DevTools by entering: empty strings, plain domain names (no scheme), FTP URLs, and valid HTTP/HTTPS URLs. All invalid inputs triggered the inline error message without making a network request. Valid inputs suppressed the error and enabled the Analyze button.

9.4 Visual Regression Testing

Chart rendering was verified by capturing screenshots after injecting known fingerprint payloads into the frontend via the browser console. Key assertions:

- Protocol pie chart renders all protocols present in the distribution with correct percentages summing to 100%.
- Histogram bars appear in the correct bucket order and heights are proportional to packet counts.
- Timeline line chart shows the correct number of data points equal to the capture duration in seconds.
- In compare mode, the overlay chart renders two distinct colored lines.

9.5 Performance Validation

The project specification requires that feature extraction and fingerprint generation complete within 3 seconds after capture, and that the total workflow completes within 20 seconds. Timing measurements taken with the synthetic capture mode (which simulates a 10-second capture

instantly) showed that `extract_features` instantly) showed that `extract_features()` and `build_fingerprint()` together complete in under 0.4seconds for a 10-second synthetic trace (~3,000 packets), well within the 3-second budget.

10. Non-Functional Requirements Compliance

NFR	Requirement	Implementation Evidence
NFR-1	User-Friendly Interface	Step-by-step stepper UI;human-readable error messages; inline validation; status panel updates during capture.
NFR-2	Fast Processing	Synthetic capture: extraction <0.4s. Live capture total:~13-15s (10s capture + 3s process). Well within 20s target.
NFR-3	Dataset Size Handling	All packet processing is in-memory via Python lists. pcap files are written to the OS temp directory. No streaming required for the academic 5 MB scope.
NFR-4	Modular Codebase	Five independent modules, each testable in isolation. No circular imports. All inter-module communication via plain Python dicts and JSON.

11.Challenges Encountered and Solutions

11.1 Root Privilege Requirement for Scapy

Problem: Scapy's sniff() function requires root/administrator privileges to open raw sockets, which are unavailable in standard university lab environments and CI systems.

Solution: The synthetic fallback mode was designed from the start as a first-class feature rather than an afterthought. The syntheticcapture() function produces statistically representative traces keyed to the target hostname, so the full pipeline can be demonstrated and tested without any special privileges. An environment variable (USE_LIVECAPTURE=1) enables the live path when privileges are available.

11.2 Scoping Capture to Target Traffic

Problem: Network sniffing captures all packets on the interface, including background OS traffic,cloud sync, update daemons, and browser activity from unrelated tabs, which would corrupt the fingerprint.

Solution: DNS resolution at the start of each analysis provides the complete set of IP addresses for the target domain. A BPF (Berkeley Packet Filter) filter of the form (host A or host B ...) or port 53 is passed to sniff(), restricting capture to packets destined for or originating from those IPs, plus all DNS traffic needed to detect query activity.

11.3 Heterogeneous Packet Representation

Problem: The pipeline must handle both real Scapy packet objects (complex, layer-aware objects) and the simpler synthetic dicts, and the extract module should not need to care which it receives.

Solution: The `_packet_view()` function in `extract.py` is a normalizer that accepts either type and returns a uniform dict with the same fields (`ts`, `size`, `proto`, `dst`, `isdns`, `dns_name`). All downstream code in `extract_features()` works only on these normalized views.

11.4 Classification Ambiguity

Problem: Many real websites do not fit neatly into one category—a news site with embedded video is neither purely 'Streaming' nor purely 'Static Content'.

Solution: Strong-signature rules handle the clearly identifiable cases with high confidence. The fallback scoring engine computes a continuous score for each category and derives confidence from the margin between the top and second-best scores. A narrow margin produces a lower confidence value, signalling to students that the classification is uncertain rather than presenting false precision.

12. Technology Stack Summary

Technology	Version	Role
Python 3.x	3.10+	Core backend language for all data processing and API logic
Flask	≥3.0	REST API server; template rendering via Jinja2; static file serving
Scapy	≥2.5	Raw packet capture and parsing; BPF filter construction
Requests	≥2.31	HTTP client for traffic generation during live capture
Chart.js	4.4.0	Client-side rendering of pie, bar, and line charts
HTML5/CSS3	-	Single-page UI structure; CSS custom properties for theming
JavaScript (ES6)	-	AJAX calls; DOM manipulation; chart data binding
pcap/JSON	-	Intermediate packet storage (pcap) and fingerprint exchange (JSON)

13.Conclusion and Learning Outcomes

The Network Fingerprint Generator and Website Behavior Profiler successfully implements all eight functional requirements and four non-functional requirements outlined in the project specification. The system delivers a complete pipeline from raw packet capture to a visual, interactive report-all accessible through a clean browser interface requiring no specialized knowledge from the end user.

From a software engineering perspective, this project demonstrated the practical value of modular design: each of the five backend modules can be tested, replaced, or extended independently. The abstraction layer in `extract.py` that normalizes both Scapy objects and synthetic dicts is a concrete example of the adapter pattern applied to real data engineering.

From a networking perspective, implementing the classifier forced a rigorous examination of what measurable properties actually distinguish different categories of web traffic. The thresholds and weights in `classify.py` encode genuine domain knowledge about packet sizes, protocol distributions, and connection patterns that are not obvious from reading protocol documentation alone.

The tool fulfills its educational purpose: a student running it against Netflix and a personal blog will immediately see-through numbers, labels, and charts-that one site transfers roughly seven times the data volume, contacts four times as many unique servers, and shows a flat, sustained traffic timeline compared to the sharp initial spike and rapid idle of the static site. This kind of direct, comparative observation is difficult to replicate from textbooks alone.

Future extensions could include PCAP export per website for Wireshark inspection, machine-learning-based classification trained on labeled captures, real-time streaming updates during capture using Server-Sent Events, and a persistent database for storing and retrieving historical fingerprints for longitudinal comparison.

References

- Scapy Project Documentation-<https://scapy.readthedocs.io>
- Flask Documentation-<https://flask.palletsprojects.com>
- Chart.js Documentation-<https://www.chartjs.org/docs/>
- Python socket module-<https://docs.python.org/3/library/socket.html>
- Wireshark BPF Filter Reference-
https://www.wireshark.org/docs/wsug_html_chunked/ChWorkBuildDisplayFilterSection.html
- RFC 1035-Domain Names (DNS)-<https://www.rfc-editor.org/rfc/rfc1035>
- RFC 793-Transmission Control Protocol-<https://www.rfc-editor.org/rfc/rfc793>
- Tanenbaum, A. S. & Wetherall, D.J.-Computer Networks, 5th Edition, Pearson, 2011.
- Kurose, J. & Ross, K.-Computer Networking: A Top-Down Approach, 8th Edition, Pearson, 2020.

Appendix A-Sample Fingerprint JSON

The following is a representative fingerprint JSON output produced by the system for a static news website:

```
{
  "site_url": "https://www.bbc.com",
  "capture_timestamp": "2025-05-07 10:32:15 UTC",
  "capture_duration_sec": 10,
  "total_packets": 1247,
  "total_bytes": 834021,
  "top_protocol": "HTTPS",
  "top_protocol_percentage": 68.4,
  "unique_ip_count": 14,
  "dns_query_count": 9,
  "mean_packet_size": 669,
  "max_packet_size": 1460,
  "min_packet_size": 64,
  "behavior_label": "Static Content",
  "confidence": 81,
  "protocol_distribution": {
    "HTTPS": 68.4, "TCP": 18.1, "DNS": 8.7, "UDP": 4.8
  },
  "packet_size_buckets": {
    "0-100": 142, "101-500": 309, "501-1000": 487,
    "1001-1500": 294, "1500+": 15
  }
}
```

Appendix B-API Request/Response Examples

B.1 Single URL Analysis

```
POST /api/analyze
Content-Type: application/json
{ "primaryUrl": "https://www.netflix.com", "captureDuration": 10 }
```

```
HTTP 200 OK
{ "siteA": { ... full fingerprint JSON ... } }
```

B.2 Comparison Analysis

```
POST /api/compare
{ "primaryUrl": "https://www.netflix.com",
  "compareUrl": "https://www.bbc.com",
  "captureDuration": 10 }
```

HTTP 200 OK


```
{ "siteA": { ... }, "siteB": { ... }, "diff": {  
  "total_bytes": { "a": 5200000, "b": 834021,  
    "who_has_more": "https://www.netflix.com", "ratio": "6.23x" },  
  "behavior_label": { "a": "Streaming", "b": "Static Content" }  
}}
```